

Análise De Algoritmos

Algoritmos e Problemas

- **Algoritmo** – Qualquer **procedimento computacional** bem definido **que toma um valor ou conjunto de valores como entrada** e **produz um valor ou conjunto de valores como saída**.
- **Problema Computacional** – Especifica em termos gerais a **relação desejada entre entrada e saída**. Assim, um **algoritmo é uma ferramenta que resolve um problema computacional**.

Exemplo – Problema de Ordenação

- Entrada: Uma sequência de n números $S = \{a_1, a_2, \dots, a_n\}$
- Saída: uma permutação de S tal que $a_1 \leq a_2 \leq \dots \leq a_n$
 - Assim, dada a entrada $\{7, 8, 4, 0, 2, 10\}$, um algoritmo de ordenação correto devolve a sequência $\{0, 2, 4, 7, 8, 10\}$ como saída.
 - Em geral, a instância de um problema consiste em uma entrada que satisfaz quaisquer restrições impostas no enunciado do problema. Portanto, sequências de diferentes tamanhos e números representam as diferentes instâncias que podem ser apresentadas a um problema de ordenação descrito acima.

Especificação de Algoritmos

- **Algoritmos podem ser especificados em linguagens de programação**, especificações formais e linguagens algorítmicas. Neste curso será utilizada uma **linguagem algorítmica conhecida como pseudocódigo**. O pseudocódigo é útil pois não tem que lidar com problemas de implementação como alocação de memória, entre outros.
- **Programa** – Uma **implementação de um algoritmo em uma linguagem de programação** (C/C++, etc).

Tipos de Problemas Computacionais

Problemas Computacionais podem ser **categorizados em quatro tipos**, a saber:

1. **Problema de decisão** – Problema onde a **saída para todas as instâncias é sim ou não**.
 - a. Exemplo: “**Dado um inteiro positivo n , n é primo?**”. A resposta pode ser **sim ou não**.

2. **Problema de busca** – Problema onde a **solução é apresentada de forma arbitrária**.
 - a. Exemplo: “**Dado um inteiro positivo n , quais são os fatores primos não triviais de n ?**”. A resposta pode ser **todos os fatores primos não triviais** (1 e n) de n . Se $n = 6$, então a **saída é $\{2, 3\}$** . Se $n = 21$, então a **saída é $\{3, 7\}$** .
3. **Problema de contagem** – Problema onde a **saída é o número de soluções para dado problema de busca**.
 - a. Exemplo: “**Dado um inteiro positivo n , quantos fatores primos não triviais n possui?**”. Para $n = 6$, saída 2 . Para $n = 27$, saída 1 .
4. **Problema de otimização** – Problema cuja **solução deve ser a melhor entre todas as possíveis soluções para determinado problema**.
 - a. Exemplo: “**Dado um conjunto de cidades e o comprimento das estradas entre todas as cidades, encontre o menor caminho possível entre duas cidades dadas, A e B** ”.

Complexidade de Problemas

Anotar depois

Complexidade de Algoritmos

Anotar depois

Conceitos Básicos de Matemática para Análise de Algoritmos

É importante compreender certas ferramentas matemáticas para praticar análise de algoritmos. A revisão a seguir trata dos conceitos matemáticos básicos para esta disciplina.

Monotonicidade

- $f(n)$ é **monótona crescente** se $m \leq n$ implica $f(m) \leq f(n)$.
- $f(n)$ é **monótona decrescente** se $m \leq n$ implica $f(m) \geq f(n)$.
- $f(n)$ é **estritamente crescente** se $m < n$ implica $f(m) < f(n)$.
- $f(n)$ é **estritamente decrescente** se $m < n$ implica $f(m) > f(n)$.

Chão e Teto

Para qualquer x real, $\lfloor x \rfloor$, o **chão de x** , corresponde ao maior inteiro menor ou igual a x . Para qualquer x real, $\lceil x \rceil$, o **teto de x** corresponde ao menor inteiro maior ou igual a x .

Exemplo de chão:

$$\lfloor 4,5 \rfloor = 4 \quad | \quad \lfloor 4 \rfloor = 4 \quad | \quad \lfloor 3,99 \rfloor = 3$$

Exemplo de teto:

$$\lceil 4,3 \rceil = 5 \quad | \quad \lceil 5 \rceil = 5 \quad | \quad \lceil 4,01 \rceil = 5$$

Séries

Dada uma sequência $a_1, a_2, a_3, \dots, a_n$, onde n é não-negativo, a soma

$a_1 + a_2 + a_3 + \dots + a_n$ pode ser escrita como $\sum_{i=1}^n a_i$

$$\text{Exemplo 1: } 1+2+3+\dots+n = \sum_{i=1}^n i$$

$$\text{Exemplo 2: } 2+4+6+8+\dots+2n = \sum_{i=1}^n 2i = 2 \sum_{i=1}^n i$$

$$\text{Exemplo 3: } 1/2+1/4+1/8+1/16+\dots+1/2^n = \sum_{i=1}^n 1/2^i$$

A forma geral é:

$$\sum_{i=1}^n a_i$$

Cada parte significa:

- $\sum \rightarrow$ **símbolo de somatória**
Indica que vários valores serão somados.
- $i \rightarrow$ **índice da soma**
É a **variável que “anda” pelos valores da sequência.**
- $i=1 \rightarrow$ **valor inicial do índice**
Diz que a soma começa quando $i=1$.
- $n \rightarrow$ **valor final do índice**
A soma termina quando $i=n$.

- $a_i \rightarrow$ termo geral

É o valor que será calculado em cada passo da soma.

Propriedades

1. $\sum_{i=1}^n C a_i = C \sum_{i=1}^n a_i$ — C é de constante
2. $\sum_{i=1}^n (a_i + b_i) = \sum_{i=1}^n a_i + \sum_{i=1}^n b_i$
3. $\sum_{i=1}^n (a_i + C) = \sum_{i=1}^n a_i + C_n$

Série Aritmética

$$1 + 2 + 3 + \dots + n = \sum_{i=1}^n i = \frac{n(n+1)}{2} = \frac{n^2+n}{2}$$

Manipulando o limite inferior da soma

- De forma geral:

○ Para $C \geq 1 \wedge C \leq n$:

$$\blacksquare \sum_{i=C}^n a_i = \sum_{i=1}^n a_i - \sum_{i=1}^{C-1} a_i$$

$$\text{Ex: } 2 + 3 + 4 + \dots + n = \sum_{i=2}^n i$$

$$\sum_{i=1}^n i = 1 + 2 + 3 + 4 + \dots + n \text{ — onde o } 2 + 3 + 4 + \dots + n \text{ é igual a } \sum_{i=2}^n i, \text{ então:}$$

$$\sum_{i=2}^n i = \sum_{i=1}^n i - \sum_{i=1}^1 i \rightarrow \sum_{i=2}^n i = \frac{n^2+n}{2} - \frac{1^2+1}{2} = \frac{n^2+n}{2} - 1$$

$$\text{Ex2: } 4 + 5 + 6 + \dots + n = \sum_{i=4}^n i$$

$$\sum_{i=1}^n i = 1 + 2 + 3 + 4 + 5 + 6 + \dots + n \text{ — onde o } 4 + 5 + 6 + \dots + n \text{ é igual a } \sum_{i=4}^n i,$$

$$\text{e } 1 + 2 + 3 \text{ é igual a } \sum_{i=1}^3 i, \text{ então:}$$

$$\sum_{i=4}^n i = \sum_{i=1}^n i - \sum_{i=1}^3 i \rightarrow \sum_{i=4}^n i = \frac{n^2+n}{2} - \frac{3^2+3}{2} = \frac{n^2+n}{2} - 6$$

Somas de Quadrados e Cubos

$$\sum_{i=1}^n i^2 = \frac{n(n+1)(2n+1)}{6}$$

$$\sum_{i=1}^n i^3 = \frac{n^2(n+1)^2}{4}$$

Série Geométrica

Para x real e $x \neq 1$, a soma:

$$\sum_{k=0}^n x^k = 1 + x + x^2 + \dots + x^n$$

É a série geométrica e tem o valor:

$$\sum_{k=0}^n x^k = \frac{x^{n+1}-1}{x-1}$$

$$\text{Ex1: } \sum_{i=0}^n 2^i = \frac{2^{n+1}-1}{2-1} = 2^{n+1} - 1$$

$$\text{Ex2: } \sum_{i=0}^{\log(n)} 2^i = \frac{2^{\log(n)+1}-1}{2-1} = 2 \cdot 2^{\log(n)} - 1$$

Série Geométrica Infinita Decrescente

Quando a **soma é infinita** e $|x| < 1$ tem a **série geométrica decrescente**:

$$\sum_{k=0}^{\infty} x^k = \frac{1}{1-x}$$

$$\text{Exemplo: } \sum_{i=0}^{\infty} (1/3)^i = \frac{1}{1-1/3} = 3/2$$

Derivando a **soma anterior** e multiplicando ambos lados por x , temos, **também para $|x| < 1$** :

$$\sum_{k=0}^{\infty} k x^k = \frac{x}{(1-x)^2}$$

$$\text{Exemplo: } \sum_{k=0}^{\infty} \frac{k}{4^k} = \sum_{k=0}^{\infty} k (1/4)^k = \frac{1/4}{(1-1/4)^2} = 4/9$$

Logaritmos

$$\log_b a \rightarrow b^x = a$$

$$\text{Ex: } \log_2 16 \rightarrow 2^x = 16$$

Propriedades

1. $\log_b b = 1$
2. $\log_b 1 = 0$
3. $\log x^y = y \log (x)$
4. $\log xy = \log (x) + \log (y)$
5. $\log x/y = \log (x) - \log (y)$
6. $x^{\log (y)} = y^{\log (x)}$
7. $\log_b a = \log_c a / \log_c b$
8. $\lg(n) = \log_2 (n)$

$$\text{Ex1: } \lg (2^n) = n \cdot \lg (2) = n$$

$$\text{Ex2: } \lg (4^{\lg(n)}) = \lg (n) \cdot \lg (4) = \lg (n) \cdot 2 = 2 \lg(n)$$

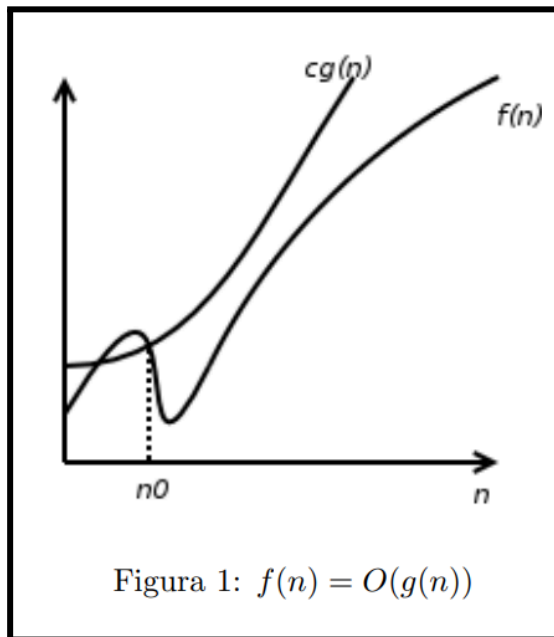
Notação Assintótica

- O **comportamento assintótico** refere-se a **quanto o custo do algoritmo aumenta conforme n cresce**.
- Se um algoritmo faz:
 - 10 operações para $n=10$
 - 100 operações para $n=100$
 - 10.000 operações para $n=10.000$
- o importante é entender a **taxa de crescimento**.

Notação O

- Chamada também de **Big O — limite superior**.
- As notações utilizadas para descrever o tempo de execução de um algoritmo são **definidas em termos de funções cujo domínio é o conjunto dos números naturais N**.
- **A notação O é usada para representar o limite assintótico superior de um algoritmo**. Para dada função $g(n)$, $O(g(n))$ é dado pelo conjunto:
$$O(g(n)) = \{f(n) : \text{existem constantes positivas } n_0 \text{ e } c \text{ tal que } f(n) \leq cg(n), \forall n \mid n \geq n_0\}$$
- Ou seja, a **notação O nos dá o limite superior para uma função, dentro de um fator constante**. Assim, **para todo n a partir de n_0 , $f(n)$ é igual ou menor ao valor de $cg(n)$** . A notação $f(n) = O(g(n))$ indica que a função $f(n)$ pertence ao conjunto $O(g(n))$, $f(n) \in O(g(n))$.

- Então, para verificar se uma função é $O(g(n))$, fazemos:
 - **$f(n) \leq cg(n)$**
- A Figura 1 mostra a representação do gráfico da notação O .



Exemplos:

1. $T(n) = n^2 + 2n$, $T(n) = O(n^2)$?
 - a. Por definição $T(n)$ é $O(n^2)$ se $T(n) \leq cn^2$ para algum $n \geq n_0$.

$$n^2 + 2n \leq cn^2$$

$$1 + 2/n \leq c$$

p/ $n = 1$ (escolhemos algum valor para testar!)

$$1 + 2 \leq c$$

$$c \geq 3$$

Assim, $T(n) = O(n^2)$ considerando $n_0 = 1$ e $c = 3$.

2. $T(n) = 3n + 2$, $T(n) = O(n^3)$?
 - a. Por definição $T(n)$ é $O(n^3)$ se $T(n) \leq cn^3$ para algum $n \geq n_0$.

$$3n + 2 \leq cn^3$$

$$3/n^2 + 2/n^3 \leq c$$

p/ $n = 1$ (escolhemos algum valor para testar!)

$$3 + 2 \leq c$$

$$c \geq 5$$

Assim, $T(n) = O(n^3)$ considerando $n_0 = 1$ e $c = 5$.

3. $T(n) = n^3 + 20n + 1$, $T(n) = O(n^2)$?
 - a. Por definição $T(n)$ é $O(n^2)$ se $T(n) \leq cn^2$ para algum $n \geq n_0$.

$$n^3 + 20n + 1 \leq cn^2$$

$$n + 20/n + 1/n^2 \leq c$$

$$c \geq n + 20/n + 1/n^2$$

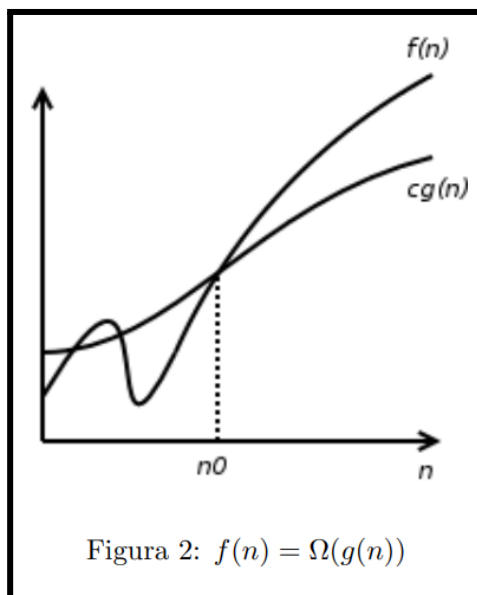
Como o **lado direito da inequação cresce infinitamente (n)**, não existe c constante. **Portanto, $T(n) \neq O(n^2)$**

Notação Ω

- Assim como a notação O provê um limite assintótico superior, a **notação Ω provê um limite assintótico inferior**. Para dada função $g(n)$, $\Omega(g(n))$ é o conjunto:

$$\Omega(g(n)) = \{f : \text{existem constantes positivas } c \text{ e } n_0 \text{ tal que } cg(n) \leq f(n), \forall n \mid n \geq n_0\}$$

- Ou seja, para **todos os valores n iguais ou maiores que n_0 o valor de $f(n)$ é maior ou igual $cg(n)$** . A notação $f(n) = \Omega(g(n))$ indica que a função $f(n)$ pertence ao conjunto $\Omega(g(n))$, $f(n) \in \Omega(g(n))$.
- Então, para verificar se uma função é $\Omega(g(n))$, fazemos:
 - $cg(n) \leq f(n)$**
- A Figura 2 mostra a representação gráfica da notação Ω .



Exemplos:

- $T(n) = n^3 + 20n$, $T(n) = \Omega(n^2)$?
 - Por definição $T(n)$ é $\Omega(n^2)$ se $cn^2 \leq T(n)$ para algum $n \geq n_0$.

$$cn^2 \leq n^3 + 20n$$

$$c \leq n + 20/n$$

p/ $n = 1$ (escolhemos algum valor para testar!)

$$c \leq 1 + 20/1$$

$$c \leq 21$$

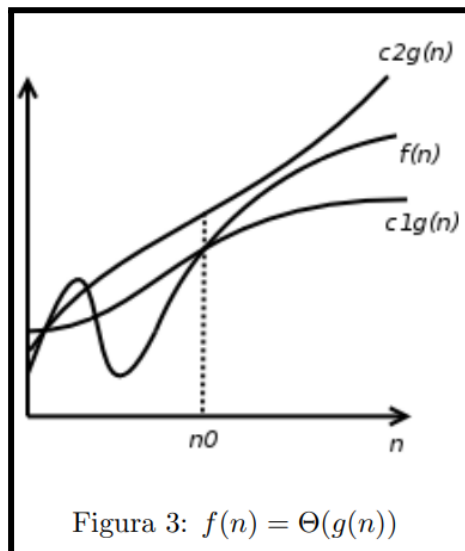
Assim, **$T(n) = \Omega(n^2)$ considerando $n_0 = 1$ e $c = 21$**

Notação Θ

- A notação Θ é usada para representar o limite assintótico ajustado de um algoritmo. Para dada função $g(n)$, $\Theta(g(n))$ é um conjunto dado por:

$$\Theta(g(n)) = \{f : \text{existem constantes positivas } c_1 \text{ e } c_2 \text{ e } n_0 \text{ tal que } c_1g(n) \leq f(n) \leq c_2g(n), \forall n \mid n \geq n_0\}$$

- Uma função $f(n)$ pertence ao conjunto $\Theta(g(n))$ se existem constantes positivas c_1 e c_2 tal que **$f(n)$ fica entre $c_1g(n)$ e $c_2g(n)$ para n suficientemente grande**. $f(n) = \Theta(g(n))$ indica que $f(n)$ pertence ao conjunto $\Theta(g(n))$, $f(n) \in \Theta(g(n))$.
- Então, para verificar se uma função é $\Theta(g(n))$, fazemos:
 - **$c_1g(n) \leq f(n) \leq c_2g(n)$**
- A Figura 3 mostra a representação gráfica da notação Θ .



○

Exemplo

1. $T(n) = \frac{1}{2}n^2 - 3n$, $T(n) = \Theta(n^2)$?
 - a. Por definição, $\frac{1}{2}n^2 - 3n = \Theta(n^2)$ se existem c_1 , c_2 e n_0 tais que $c_1n^2 \leq \frac{1}{2}n^2 - 3n \leq c_2n^2$ para todo n a partir de n_0 . Assim,

$$c_1 \leq \frac{1}{2} - \frac{3}{n} \leq c_2$$

p/ o lado direito

$$\frac{1}{2} - \frac{3}{n} \leq c_2$$

$$n = 1$$

$$\frac{1}{2} - \frac{3}{1} \leq c_2$$

$$c_2 \geq \frac{5}{2}$$

p/ o lado esquerdo

$$c_1 \leq \frac{1}{2} - \frac{3}{n}$$

$$n = 7$$

$$c_1 \leq \frac{1}{2} - \frac{3}{7}$$

$$c_1 \leq \frac{1}{14}$$

Assim, considerando $n_0 = 7$, $c_1 = 1/14$ e $c_2 = 5/2$ é possível concluir que $\frac{1}{2} n^2 - 3n = \Theta(n^2)$.

Análise Assintótica de Trechos Iterativos

- **Trecho Iterativo:** Um trecho iterativo é uma sequência de instruções, sem recursões.
- **Algoritmo Iterativo:** Um Algoritmo iterativo é um algoritmo composto de uma sequência de trechos iterativos. Consideramos que todo algoritmo iterativo também é um trecho iterativo, uma vez que na prática eles são apenas sequências de instruções.
- Conforme discutimos anteriores, **não é necessário contabilizar exatamente o custo de cada linha de um algoritmo**, uma vez que **assintoticamente o custo será definido pelo termo de maior ordem**.
- Desta forma, é suficiente **analisarmos o custo apenas dos trechos mais custosos de um algoritmo**. Chamamos estes trechos de **trechos quentes**.
 - **Como saber quais trechos são mais custosos que outros?**
 - A idéia é aprender o custo de diferentes padrões e usar a experiência adquirida para classificar trechos em potencial como custosos ou não.
 - Por exemplo, **trechos que estão em um nível mais profundo de aninhamento de laços de repetição são sempre suspeitos de serem mais custosos que trechos que estão fora de laços de repetição**.

Trecho 1

```

1: Soma = 0
2: for i = 1; i ≤ n; i ++ do
3:   for j = 1; j ≤ n; j ++ do
4:     Soma ++
5:   end for
6: end for

```

Análise

A variável que indica o tamanho do problema é n . Note que a linha 4 é a **linha mais frequentemente executada do algoritmo**. Portanto, **avaliar o número de execuções da linha 4 é suficiente para determinarmos o comportamento assintótico da função de custo do trecho**. Note também que a quantidade de vezes que a linha 4 é executada depende de ambos os laços for que a contém. Como a **linha 4 não depende de n , ou seja seu custo é constante**, consideramos que tem **complexidade $\Theta(1)$** .

Note que a **variável de controle do laço interno (j) não depende da variável de controle do laço externo (i)**. Portanto, podemos **analisar o laço interno independentemente do laço externo**. Como o laço interno **executa n vezes a linha 4**, sua complexidade é dada por:

$$\sum_{i=1}^n 1 = n = \Theta(n)$$

Assim, em termos de custo, podemos analisar o Trecho 1 como:

```
1: Soma = 0
2: for i = 1; i ≤ n; i++ do
3:     //
4:     //Θ(n)
5:     //
6: end for
```

Notamos, portanto, que o **laço externo execute n vezes o laço interno que, conforme mostrado, tem complexidade $\Theta(n)$** . Assim, a **complexidade do algoritmo é dada por:**

$$n \Theta(n) = \Theta(n^2)$$

Portanto, a **complexidade deste trecho iterativo é $\Theta(n^2)$** .

Método alternativo:

1. Após descobrir sobre a linha 4 e sua complexidade **$\Theta(1)$**
2. Analisar o laço externo (i):
 - a. Executa n vezes
3. Analisar o laço interno (j):
 - a. Executa n vezes
4. Então o total é:
 - a. Vamos na somatória e colocamos:
 - i. **“Em cima” do Σ colocamos a quantidade de execuções do laço externo (n)**

- ii. “Do lado” do Σ colocamos a quantidade de execuções do laço interno (n)

$$\text{b. } \sum_{i=1}^n n = n * n = n^2$$

c. Logo: $\Theta(n^2)$

Trecho 2

```
1: Soma = 0
2: for i = 1; i ≤ n; i ++ do
3:   for j = 1; j ≤ i; j ++ do
4:     Soma ++
5:   end for
6: end for
```

Análise

Como a linha 4 não depende de n, ou seja seu custo é constante, consideramos que tem complexidade $\Theta(1)$.

1. Laço externo:

a. Executa n vezes

2. Laço interno:

a. Executa i vezes

3. Total será:

a. Em cima” do Σ colocamos a quantidade de execuções do laço externo (n)

b. “Do lado” do Σ colocamos a quantidade de execuções do laço interno (i)

$$\text{c. } \sum_{i=1}^n i = \frac{n(n+1)}{2} = \frac{n^2+n}{2}$$

d. Logo: $\Theta(n^2)$

Trecho 3

```
1: Soma = 0
2: for k = 1; k ≤ n; k* = 2 do
3:   for j = 1; j ≤ n; j ++ do
4:     Soma ++
5:   end for
6: end for
```

Análise

Como a **linha 4 não depende de n**, ou seja seu **custo é constante**, consideramos que tem **complexidade $\Theta(1)$** .

1. Laço externo:

- i. O k dobra a cada iteração:
- ii. 1, 2, 4, 8, 16
- iii. Queremos saber quantas vezes ele dobra até chegar em n.
- iv. Isso equivale a: $2^x \leq n$
- v. Aplicando log: $x \leq \log_2 n$
- vi. Então:

b. Executa **$\lg(n)$** vezes

2. Laço interno:

a. Executa n vezes

3. Total será:

a. **Em cima” do Σ** colocamos a **quantidade de execuções do laço externo ($\lg(n)$)**

b. **“Do lado” do Σ** colocamos a **quantidade de execuções do laço interno (n)**

c. $\sum_{i=1}^{\lg(n)} n = n * \lg(n)$

d. Logo: **$\Theta(n \log n)$**

Trecho 4

1: Soma = 0

2: **for** k = 1; k $\leq$ n; k* = 2 **do**

3: **for** j = 1; j $\leq$ k; j + + **do**

4: Soma + +

5: **end for**

6: **end for**

Análise

Como a **linha 4 não depende de n**, ou seja seu **custo é constante**, consideramos que tem **complexidade $\Theta(1)$** .

1. Laço externo:

a. Executa **$\lg(n)$** vezes

2. Laço interno:

a. Executa k vezes

- i. Para cada valor de k, o laço interno executa k vezes.
- ii. Então o total é:

- iii. $1+2+4+8+\dots+n$
- iv. Isso é uma Progressão Geométrica (PG).
- v.

3. Total será:

a. Como é um Série Geométrica:

- i. **Em cima” do Σ colocamos a quantidade de execuções do laço externo ($\lg(n)$)**
- ii. **“Do lado” do Σ colocamos a quantidade de execuções do laço interno, que em cada iteração é dada por 2 já que a variável k dobra a cada passo e eleva esse 2 a i.**

$$b. \sum_{i=0}^{\lg(n)} 2^i = \frac{2^{\lg(n)+1} - 1}{2 - 1} = 2 \cdot 2^{\lg(n)} - 1 = 2 \cdot n^{\lg(2)} - 1 = 2n - 1$$

c. Logo: $\Theta(n)$

Trecho 5

```
1: if n é par then
2:   F1(n)
3: else
4:   F2(n)
5: end if
```

Considerando que $F1(n) = \Theta(n)$ e $F2(n) = n \lg(n)$, qual a complexidade deste trecho?

A variável que indica o tamanho do problema é n . No **melhor caso**, n é par e **F1 é executada**, portanto a complexidade é $\Theta(n)$. Por outro lado, no **pior caso**, n é ímpar e **F2 é executada**, portanto a complexidade é $\Theta(n \lg(n))$. Note que a complexidade da linha 1 é constante, portanto não influencia no comportamento assintótico da função de custo.

Divisão e Conquista

- Técnica de projeto de algoritmos
- Baseia-se em **dividir um problema em subproblemas menores recursivamente**. Desta forma, **quando o problema se torna pequeno suficiente uma solução simples pode ser aplicada**. As soluções dos problemas menores são combinadas para obter a solução final do problema original.
 1. **Dividir** o problema em um número de subproblemas que são instâncias menores do mesmo problema;

2. **Conquistar** os subproblemas, resolvendo-os recursivamente.
Se os subproblemas são pequenos o suficiente, podem ser resolvidos de forma trivial;
3. **Combinar** as soluções dos subproblemas para obter o resultado final.

MergeSort

- O algoritmo de ordenação por intercalação, o **MergeSort**, segue este paradigma. Intuitivamente:
 - **Dividir** a sequência de n elementos em 2 subsequências com $n/2$ elementos.
 - **Conquista** ordenar as duas subsequências recursivamente utilizando MergeSort.
 - **Combinar**: intercalar as subsequências ordenadas para produzir o resultado final ordenado.
- A principal operação no MergeSort é o procedimento **Merge**, que é responsável por intercalar 2 subsequências ordenadas, produzindo uma sequência ordenada como saída. O algoritmo é dado a seguir:

```

1: procedure MERGE(V, p, q, r)
2:    $n_1 = q - p + 1$ 
3:    $n_2 = r - q$ 
4:    $L[1..n_1 + 1]$ 
5:    $R[1..n_2 + 1]$ 
6:   for  $i = 1; i \leq n_1; i++$  do
7:      $L[i] = V[p + i - 1]$ 
8:   end for
9:   for  $j = 1; j \leq n_2; j++$  do
10:     $R[j] = V[q + j]$ 
11:  end for
12:   $L[n_1 + 1] = \infty$ 
13:   $R[n_2 + 1] = \infty$ 
14:   $i = 1$ 
15:   $j = 1$ 
16:  for  $k = p; k \leq r; k++$  do
17:    if  $L[i] \leq R[j]$  then
18:       $V[k] = L[i]$ 
19:       $i++$ 
20:    else
21:       $V[k] = R[j]$ 
22:       $j++$ 
23:    end if
24:  end for
25: end procedure

```

▷ L e R são vetores dinâmicos

Análise

- Seja $n = r - p + 1$. As linhas 2–5 e 12–15 executam em tempo constante. O laço for das linhas 6–8 executa $n_1 = q - p + 1$ vezes e o laço for das linhas 9–11 executa $n_2 = r - q$ vezes. Portanto o custo do trecho das linhas 6–11 tem custo $n_1 + n_2 = (q - p + 1) + (r - q) = r - p + 1 = n$

= $\Theta(n)$. O laço das linhas 16–24 repete n vezes com operações de tempo constante, portanto, $\Theta(n)$. Assim, o tempo total do procedimento Merge é $\Theta(n) + \Theta(n) + \Theta(1) = \Theta(n)$.

- O passo da conquista ordena recursivamente dois subproblemas, cada um com a metade dos elementos do problema atual. Os passos de divisão e conquista são dados por:

```
1: procedure MERGESORT(V, p, r)
2:   if  $p < r$  then
3:      $q = \lfloor \frac{p+r}{2} \rfloor$ 
4:     MergeSort(V, p, q)
5:     MergeSort(V, q + 1, r)
6:     Merge(V, p, q, r)
7:   end if
8: end procedure
```

$\triangleright \Theta(n)$

- O custo deste algoritmo é dado por:

$$T(n) = \begin{cases} \Theta(1) & \text{se } n \leq 1 \\ 2T(\frac{n}{2}) + \Theta(n) & \text{Se } n > 1 \end{cases}$$

- Note que a equação é expressa em função dela mesma com subproblemas menores. Este tipo de equação é denominada equação de recorrência, ou simplesmente, recorrência.
- Resolver recorrências é uma arte. Embora algumas formas de recorrências possuam soluções conhecidas, como o método mestre, a solução de recorrências é realizada de forma “artesanal”. No entanto, serão apresentadas técnicas que auxiliam a resolução de recorrências:
 - Método da Substituição
 - Árvore de Recursão
 - Método Mestre

Método da Substituição

- O método da substituição é utilizado para determinar limites superiores e inferiores para recorrências. O método é baseado na indução matemática e é realizado em dois passos.
 1. Chutar um limite superior ou inferior para uma recorrência.
 2. Usar indução matemática para encontrar as constantes, conforme as definições da notação assintótica, para mostrar que o limite vale.

Quando usar?

- Normalmente em recorrências como:
 - $T(n) = 2T(n/2) + n$

- $T(n)=T(n-1)+1$
- $T(n)=T(n/2)+n$
- e outras semelhantes.

Passo a passo

Passo 1: Chutar a resposta

Você observa a recorrência e propõe uma complexidade.

Exemplo:

$$T(n)=T(n-1)+1$$

Você pode suspeitar que:

$$T(n)=O(n)$$

Passo 2: Assumir que a hipótese é verdadeira

Suponha que para valores menores:

$$T(k) \leq ck$$

onde c é uma constante positiva.

OBS:

- **Caso peça para demonstrar que a recorrência seja O , ex: $T(n)=O(n)$**
 - Fazer a hipótese como: $T(k) \leq ck$
- **Caso peça para demonstrar que a recorrência seja Ω , ex: $T(n)=\Omega(n)$**
 - Fazer a hipótese como: $T(k) \geq dk$
- **Caso peça para demonstrar que a recorrência seja Θ , ex: $T(n)=\Theta(n)$**
 - Fazer ambas hipóteses, ou seja tanto:
 - $T(k) \leq ck$ e
 - $T(k) \geq dk$

Passo 3: Substituir na recorrência

Como

$$T(n)=T(n-1)+1$$

substituímos:

$$T(n) \leq c(n-1)+1$$

Passo 4: Simplificar

$$T(n) \leq cn - c + 1$$

Queremos mostrar que:

$$T(n) \leq cn$$

Isso acontece se:

$$-c+1 \leq 0$$

ou seja:

$$c \geq 1$$

Logo:

$$T(n) = O(n)$$

Mudança de Variáveis

Exemplos

Árvore de Recursão

- Embora o método da substituição forneça uma prova sucinta do limite assintótico para uma recorrência, pode ser difícil achar um palpite exato para o limite ajustado para uma função de custo. Uma ferramenta útil para facilitar o palpite do limite assintótico é a **árvore de recursão**. Em uma árvore de recursão, **cada nó representa o custo de um subproblema de tamanho específico**. Podemos **somar o custo de cada nível da árvore para obter o custo por nível**, para **então somar o custo de todos os níveis e obter o custo total**.
- Normalmente a árvore de recursão é **utilizada apenas para obter um palpite inicial que depois é verificado utilizando o método da substituição**. Portanto, alguns detalhes podem ser omitidos uma vez que a prova do limite pode ser realizada com o método da substituição. No entanto, se o método for aplicado rigorosamente, é possível usar a árvore como prova direta de limites assintóticos ajustados.

Quando usar?

A **Árvore de Recursão** é uma das técnicas mais usadas em Análise de Algoritmos para resolver recorrências, especialmente as do tipo:

$$T(n) = aT(nb) + f(n)$$

Ela funciona visualizando cada chamada recursiva como um nó de uma árvore e somando o custo de todos os níveis.

Passo a Passo

Para uma recorrência:

$$T(n)=aT(n/b)+f(n)$$

Faça:

1. Desenhe a árvore

Identifique:

- Quantos filhos cada nó gera (a)
- Tamanho de cada filho (n/b)

2. Calcule o custo do nível i

Normalmente:

$$a^i \cdot f\left(\frac{n}{b^i}\right)$$

3. Ache a altura

Resolva:

$$\frac{n}{b^h} = 1$$

obtendo:

$$h = \log_b n$$

Recorrência	Altura
$T(n)=T(n/2)$	$\log_2 n$
$T(n)=T(n/3)$	$\log_3 n$
$T(n)=T(7n/10)$	$\log_{10/7} n$
$T(n)=T(n-1)$	$n - 1$
$T(n)=T(n-2)$	$(n-1)/2$
$T(n)=T(\alpha n)$	$\log_{1/\alpha} n$
$T(n)=T(n/b)$	$\log_b n$

Na divisão: $T(n) = T(7n/10) + n$

- Qual é o tamanho do problema no nível i ? É $n \cdot \left(\frac{7}{10}\right)^i$.
- O que nós fazemos? Igualamos esse tamanho a 1 (caso base):

$$n \cdot \left(\frac{7}{10}\right)^i = 1$$

- Aqui, para isolar o i , nós precisamos usar **logaritmo**, porque o i está lá em cima no expoente.

Na subtração: $T(n) = T(n - 2) + n$

- Qual é o tamanho do problema no nível i ? É $n - 2i$.
- O que nós fazemos? Igualamos esse tamanho a 1 (caso base):

$$n - 2i = 1$$

- Aqui, para isolar o i , nós **não** precisamos de logaritmo, porque o i está no chão, multiplicando o 2. É uma equação simples do primeiro grau!

Resumo

Em nenhum dos casos nós igualamos a altura ao "valor do nível i ". Nós sempre fazemos:

Tamanho do problema no nível i = Caso Base (1)

A única diferença é a álgebra para isolar o i depois disso:

- Se o problema **divide**, o i vira expoente $\implies$ vira **Logaritmo**.
- Se o problema **subtrai**, o i vira multiplicador $\implies$ vira **Frações/Função Linear**.

4. Some os níveis

$$T(n) = \sum_{i=0}^{\log_b n} [a^i f(\frac{n}{b^i})]$$

$$T(n) = \sum_{i=0}^{\text{altura}} \text{custo no nível } i$$

5. Simplifique para Θ

Observe se:

- **Todos os níveis têm o mesmo custo?**
 - Ex: $n, n, n, n, \dots$
 - **Multiplica pelo número de níveis.**
 - $\Theta(n \log n)$
- **Os custos diminuem?**
 - $n, n/2, n/4, n/8, \dots$
 - **O topo domina.**
 - $\Theta(\text{primeiro nível})$
- **Os custos aumentam?**
 - Ex: $n, 2n, 4n, 8n, \dots$
 - **As folhas dominam.**
 - $\Theta(\text{último nível})$
- **Surge uma progressão geométrica?**
 - $a + ar + ar^2 + \dots$
 - Analise a razão:
 - $r < 1 \rightarrow$ **topo domina**
 - $r = 1 \rightarrow$ **todos os níveis iguais**
 - $r > 1 \rightarrow$ **folhas dominam**

Exemplos

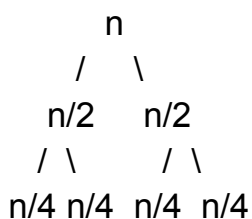
1. Considere a recorrência:

$$T(n) = 2T(n/2) + n$$

Ela representa um algoritmo que:

- Divide o problema em 2 subproblemas.
- Cada subproblema tem tamanho $n/2$.
- O trabalho fora da recursão custa n .

A árvore fica assim:



Cada nó mostra apenas o custo não-recursivo.

Passo 1 – Desenhe os primeiros níveis

Raiz:

None

Custo = n

Primeiro nível:

None

2 nós

custo de cada um = $n/2$

Segundo nível:

None

4 nós

custo de cada um = $n/4$

Passo 2 – Descubra o custo de cada nível

Nível 0

1 nó

n

Nível 1

2 nós

$2 \cdot n/2 = n$

Nível 2

4 nós

$4 \cdot n/4 = n$

Nível 3

8 nós

$8 \cdot n/8 = n$

Perceba:

Custo por nível = n

Passo 3 – Descubra a altura da árvore

Pergunta:

“Quantas vezes posso dividir n por 2 até chegar em 1?”

$$\frac{n}{2^h} = 1$$

Logo:

$$n = 2^h$$

Aplicando log:

$$h = \log_2 n$$

A árvore possui:

$$\log n + 1$$

níveis.

Passo 4 – Somar todos os níveis

Como cada nível custa n :

$$T(n) = n + n + n + \dots$$

Existem $\log n + 1$ níveis

Então:

$$T(n) = n(\log n + 1)$$

$$T(n) = \Theta(n \log n)$$

Método Mestre

- O **Método Mestre (Master Theorem)** é uma das ferramentas mais usadas em Análise de Algoritmos para resolver recorrências da forma:

$$T(n) = aT(n/b) + f(n)$$

- onde:
 - **a** = número de subproblemas gerados
 - **b** = fator de redução do tamanho do problema
 - **f(n)** = custo fora das chamadas recursivas (dividir, combinar, etc.)

Os 3 Casos

1. Se $f(n) = O(n^{\log_b(a) - \epsilon})$ para $\epsilon > 0$ constante, então $T(n) = \Theta(n^{\log_b a})$;
2. Se $f(n) = \Theta(n^{\log_b a})$, então $T(n) = \Theta(n^{\log_b a} \lg n)$;
3. Se $f(n) = \Omega(n^{\log_b(a) + \epsilon})$ para $\epsilon > 0$ constante, e se $af(n/b) \leq cf(n)$ para $c < 1$ constante, e n suficientemente grande, então $T(n) = \Theta(f(n))$.

Passo a passo

1. Identifique:
 - a. $a, b, f(n)$
2. Calcule:
 - a. $n^{\log_b a}$
3. Compare $f(n)$ com $n^{\log_b a}$
 - a. Pergunte:
 - i. É menor? → Caso 1
 - ii. É igual? → Caso 2
 - iii. É maior? → Caso 3
4. Escreva a resposta usando o caso encontrado.

Exemplo

Resolva:

$$T(n)=8T(n/2)+n^2$$

Passo 1

$$a=8$$

$$b=2$$

$$f(n)=n^2$$

Passo 2

$$n^{\log_2 8}$$

$$n^3$$

Passo 3

Comparação:

$$n^2 < n^3$$

Logo:

Caso 1.

Passo 4

Resposta:

$$T(n) = \Theta(n^3)$$

Prova de Correção por Invariante de Laço

- A **prova de correção por invariante de laço** é uma das técnicas mais importantes em **Análise de Algoritmos**, principalmente para provar que **algoritmos iterativos** (com **for**, **while**) realmente fazem o que prometem.

O que é uma Invariante?

- Um **invariante de laço** é uma **propriedade que permanece verdadeira durante toda a execução do laço.**
- A ideia é:
 - **"Antes de cada iteração do laço, alguma afirmação específica é verdadeira."**
- Se conseguirmos provar isso, podemos demonstrar que o algoritmo está correto.

As Três Etapas da Prova

- Toda prova por invariante possui **três partes**:
 - 1. Inicialização**
 - a. Você prova que o invariante é verdadeiro antes da primeira execução do laço.
 - 2. Manutenção**
 - a. Você assume que o invariante é verdadeiro em uma iteração e mostra que ele continua verdadeiro na próxima.
 - 3. Término**
 - a. Quando o laço termina, você usa o invariante para provar que o resultado produzido está correto.

Exemplos

Busca do Maior Elemento

Considere:
maior = A[0];

```
for (i = 1; i < n; i++) {  
    if (A[i] > maior)  
        maior = A[i];  
}
```

Queremos provar que ao final maior contém o maior elemento do vetor.

Passo 1: Encontrar o Invariante

Pergunte: O que é verdade antes de cada repetição?

Aqui: **Antes da interação i**, a variável **maior** contém o maior elemento entre A[0] e A[i-1];
Esse será nosso invariante.

Passo 2: Inicialização

Antes da primeira interação:

maior = A[0]

i = 1

O maior elemento do conjunto: A[0]

Logo, o invariante é verdadeira.

Passo 3: Manutenção

Assuma que **antes da interação i**: maior contém o maior elemento entre A[0] e A[i-1].

Durante a interação existem dois casos:

Caso 1

A[i] <= maior

Nada muda.

Então **maior** continua sendo o maior entre:

A[0] ... A[i]

Caso 2

A[i] > maior

Atualizamos:

$\text{maior} = A[i]$

Agora maior passa a ser o maior entre:

$A[0] \dots A[i]$

Portanto, o invariante continua válido.

Passo 4: Término

O laço termina quando:

$i = n$

Pelo invariante: **maior** contém o maior elemento entre $A[0]$ e $A[n-1]$

Ou seja, o algoritmo está **correto**.

Insertion Sort

Considere:

```
for (j = 1; j < n; j++) {
```

```
    chave = A[j];
```

```
    i = j - 1;
```

```
    while (i >= 0 && A[i] > chave) {
```

```
        A[i+1] = A[i];
```

```
        i--;
```

```
    }
```

```
    A[i+1] = chave;
```

```
}
```

O invariante tradicional é: Antes da iteração j , o subvetor $A[0..j-1]$ está ordenado.

Inicialização

Antes de $j=1$: **$A[0]$**

Um vetor com um elemento já está ordenado.

Manutenção

Suponha que: $A[0..j-1]$

está ordenado.

A iteração insere $A[j]$ na posição correta.

Após isso: $A[0..j]$
fica ordenado

Término

Quando: $j = n$

temos: $A[0..n-1]$
ordenado

Logo, o algoritmo está correto.

HeapSort

- O tempo de execução do Heapsort é $O(n \lg n)$. No entanto o Heapsort é considerado mais eficiente que o MergeSort pois realiza a ordenação in-place, ou seja, **somente uma quantidade constante de elementos são mantidos fora do vetor sendo ordenado**. O Heapsort também introduz um conceito interessante: utilizar uma estrutura de dados especial para organizar informações sobre a ordenação: **heaps**.

Heaps

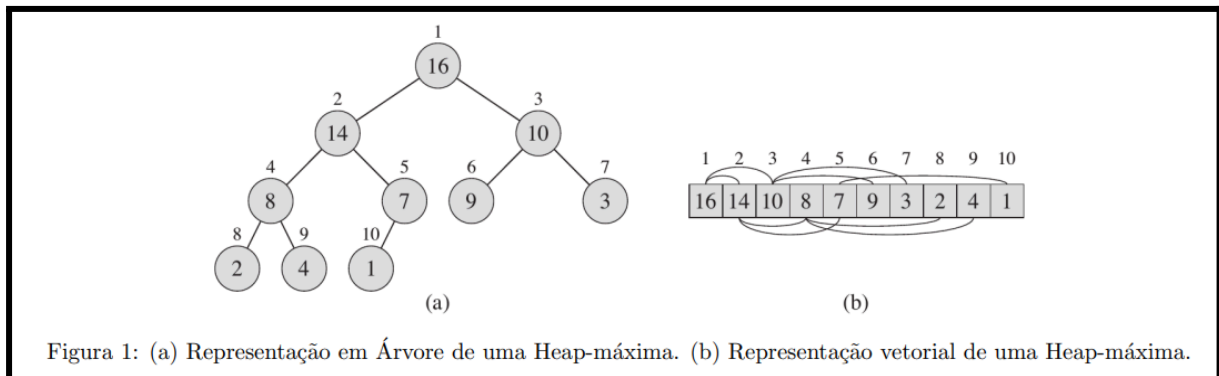
- Uma **heap** é um **vetor que pode ser interpretado como uma árvore binária quase completa**. Cada nó da árvore corresponde a uma posição no vetor. Esta árvore é completa em todos os níveis, exceto o último, que é preenchido da esquerda para a direita. A **raiz da árvore é $A[1]$** . Dado um índice i de um nó, é possível computar os pais e os filhos de i :

$$\begin{aligned} \text{PAI}(i) &= \left\lfloor \frac{i}{2} \right\rfloor \\ \text{ESQUERDO}(i) &= 2i \\ \text{DIREITO}(i) &= 2i + 1 \end{aligned}$$

- que podem ser computados com apenas uma instrução em computadores eletrônicos modernos.
- Existem **dois tipos de heaps binários**: **heap-máximo** e heap-mínimo. Em ambos tipos os **valores dos nós satisfazem uma propriedade**

heap que depende do tipo da heap. Em um heap-máximo a propriedade heap é:

- “Para cada nó i , exceto a raiz, $A[\text{PAI}(i)] \geq A[i]$. Alternativamente, $\forall x (i \neq 1 \rightarrow A[\text{PAI}(i)] \geq A[i])$ ”
- Ou seja, **o valor de um nó é, no máximo, o valor de seu pai**. Desta forma, **o maior elemento em um heap-máximo está na raiz** e a subárvore de dado nó i contém elementos que não são maiores que o valor em i .



- No algoritmo **Heapsort são utilizados heap-máximos**. Quando uma heap é interpretada como uma árvore, **a altura de um nó é o número de arestas no caminho mais longo de um nó até uma folha**. A **altura da heap é a altura da raiz**. Como uma heap de n elementos é função de uma árvore binária completa, sua **altura é $\Theta(\lg n)$** . Além disto, as **operações binárias em uma árvore heap** executam em tempo proporcional à altura da árvore, **portanto executam em tempo $\Theta(\lg n)$** . Estes são alguns fatos que garantem o bom desempenho do Heapsort, como será estudado mais adiante. As funções a seguir são utilizadas no Heapsort.
- **MAX-HEAPIFY**, executa em **$\Theta(\lg n)$** e é **responsável por manter a propriedade heap-máximo**;
- **BUILD-MAX-HEAP**, executa em **$\Theta(n)$** , e **converte um vetor desordenado em uma heap-máxima**; e
- **HEAPSORT**, que executa em **$\Theta(n \lg n)$** e **ordena o vetor explorando a propriedade heap-máximo**.

MAX-HEAPIFY

- A função **MAX-HEAPIFY** é responsável por manter a propriedade heap-máximo. As entradas são um vetor **A**, um índice **i** e o tamanho da heap **th**. A função assume que as árvores binárias em **ESQUERDO(i)** e **DIREITO(i)** são heaps máximos, mas **A[i]** pode ser menor que **ESQUERDO(i)** e **DIREITO(i)**, o que fere a propriedade heap-máximo. **th** indica qual é o último elemento de **A** que faz parte da heap. **MAX-HEAPIFY** arruma a propriedade da árvore.

```
MAX-HEAPIFY(A, i, th)
1. l = ESQUERDO(i)
2. r = DIREITO(i)
3. IF l <= th AND A[l] > A[i]
4.     largest = l
5. ELSE largest = i
6. IF r <= th AND A[r] > A[largest]
7.     largest = r
8. IF largest <> i
9.     trocar A[i] e A[largest]
10.    MAX-HEAPIFY(A, largest, th)
```

- **MAX-HEAPIFY** é um procedimento recursivo que funciona da seguinte maneira:
 - O maior entre **A[1]**, **A[ESQUERDO(i)]** e **A[DIREITO(i)]** é determinado.
 - Se **A[i]** já é o maior, então a propriedade heap-máximo já está OK. Caso contrário, um dos filhos é maior que **A[i]**. Então **A[i]** é trocado com o maior dos filhos, arrumando a propriedade localmente.
 - Após a troca, o nó que estava com o maior valor agora tem o valor que o nó **i** tinha. Desta forma, este não pode violar a propriedade heap-máximo daquele índice em diante. A sub-árvore é arrumada recursivamente.
- O tempo de execução de **MAX-HEAPIFY** em uma subárvore de altura **n** a partir do nó **i** é $\Theta(1)$ para arrumar a relação entre **A[1]**, **A[ESQUERDO(i)]** e **A[DIREITO(i)]** mais o tempo necessário para executar **MAX-HEAPIFY** em uma subárvore de **i** (chamada recursiva). A altura das subárvores possuem no máximo $2n/3$ dos nós da árvore, e

acontece quando o último nível está cheio até a metade. **Portanto, podemos descrever o tempo de MAX-HEAPIFY pela recorrência**

$$T(n) \leq T\left(\frac{2n}{3}\right) + \Theta(1)$$

- Pelo método mestre, esta recorrência é **$T(n) = O(\lg n)$** . Como a altura h de uma árvore heap com n elementos tem altura $\lg n$, o tempo de execução de MAX-HEAPIFY pode ser descrito por **$O(h)$** .

BUILD-MAX-HEAP

- **BUILD-MAX-HEAP** pode ser utilizado para converter um vetor de $A[1..n]$ em uma heap-máxima. Sabemos que os elementos no subvetor $A[\lfloor n/2 \rfloor + 1 \dots n]$ são folhas, que são heaps unitárias. Portanto, **basta percorrer os demais nós da árvore e executar MAX-HEAPIFY em cada um deles em ordem inversa.**

```
BUILD-MAX-HEAP(A, n)
1. FOR i = CHAO(n/2) DOWNT0 1 DO
2.   MAX-HEAPIFY(A, i, n)
```

- Para provar que este algoritmo converte um vetor $A[1 \dots n]$ em uma heap-máxima, utilizamos a seguinte invariante de laço:
 - “No início de cada iteração do laço FOR das linhas 1–2, cada nó $i + 1, i + 2, \dots, n$ é raiz de uma heap-máxima”.
- **INICIALIZAÇÃO:** Antes da primeira iteração, $i = \lfloor n/2 \rfloor$. Cada nó $\lfloor n/2 \rfloor + 1, \lfloor n/2 \rfloor + 2, \dots, n$ é folha, e portanto, a raiz de uma heap-máxima trivial.
- **MANUTENÇÃO:** Note que os índices dos filhos de i são maiores que i . Pela invariante de laço, estes filhos são ambos raízes de heaps máximos, que é a condição necessária para invocar MAX-HEAPIFY para fazer i tornar-se uma heap máxima. Além disto, MAX-HEAPIFY mantém a propriedade que os nós $i + 1, i + 2, \dots, n$ são todos raízes de heaps-máximos. **Ao decrementar i no laço FOR, a invariante de laço é restabelecida para a próxima iteração.**
- **TÉRMINO:** No termino, $i = 0$. Pela invariante de laço, cada nó $1, 2, \dots, n$ é a raiz de uma heap máxima. Portanto, após a última iteração, todos os elementos no subvetor $A[i + 1, i + 2, \dots, n] = A[1, 2, \dots, n]$, são raízes de heap-máximo, e inclui todos os elementos do vetor

A. Por definição, **um vetor é heap máxima se todos os elementos do vetor são raízes de heap-máximo**. Como mostramos que isto é verdade ao final da execução de BUILD-MAX-HEAP, **o vetor A é um heap-máximo**.

Desempenho de BUILD-MAX-HEAP

- O limite assintótico superior de BUILD-MAX-HEAP pode ser estimado da seguinte forma: **cada chamada a MAX-HEAPIFY tem custo $O(\lg n)$ e BUILD-MAX-HEAP faz $O(n)$ chamadas delas**. Desta forma, $T(n) = O(n) * O(\lg n) = O(n \lg n)$ **descreve o limite assintótico superior**. Contudo, embora correto, este limite não está ajustado o máximo possível.
- O limite assintótico superior de BUILD-MAX-HEAP pode ser limitado por:

$$O\left(n \sum_{h=0}^{\lfloor \lg n \rfloor} \frac{h}{2^h}\right) = O\left(n \sum_{h=0}^{\infty} \frac{h}{2^h}\right) = O(2n) = O(n).$$

HEAPSORT

```

HEAPSORT(A, n)
1. BUILD-MAX-HEAP(A, n)
2. th = n
3. FOR i = n DOWNTO 2 DO
4.     troca A[1] e A[i]
5.     th = th - 1
6.     MAX-HEAPIFY(A, 1, th)
    
```

- O algoritmo **HEAPSORT** **inicia com a construção de uma heap máxima a partir do vetor original**. Como o elemento máximo do vetor está em $A[1]$, é possível trocá-lo com $A[n]$, sua posição correta. Ao descartar o último elemento da heap e mantendo uma variável de controle junto ao vetor heap indicando o final da heap, podemos verificar que os **filhos da raiz mantêm-se heaps máximos**. No entanto, o **novo elemento raiz pode violar a propriedade heap-máximo**. Portanto, **basta restaurar sua propriedade usando MAX-HEAPIFY**, obtendo uma nova uma heap-máxima $A[1, \dots, i - 1]$.

O algoritmo repete este processo até obter uma heap de tamanho 2.

- Heapsort invoca **BUILD-MAX-HEAP** que executa em $\Theta(n)$ e que cada uma das $n-1$ chamadas a **MAX-HEAPIFY** executa em tempo $\Theta(\lg n)$. Assim, **o tempo de execução de HEAPSORT é dado pela função**

$$T(n) = \underbrace{(n-1) \cdot \Theta(\lg n)}_{\text{Linhas 3-6}} + \underbrace{\Theta(n)}_{\text{BUILD-MAX-HEAP}} = \Theta(n \lg n).$$

QuickSort

- Embora o **pior caso do QUICKSORT** seja $\Theta(n^2)$ para um **vetor com n elementos**, é o algoritmo mais prático para ser usado por executa em **tempo $\Theta(n \lg n)$ no caso médio**. Além disto, os fatores constantes escondidos pela notação assintótica em $\Theta(n \lg n)$ **são normalmente menores que os encontrados nos algoritmos HEAPSORT e MERGESORT**. Além disto, ao contrário de alguns algoritmos de ordenação, a ordenação acontece in loco e funciona bem até em ambientes com memória virtual, indispensáveis hoje em dia.

O Algoritmo

```
QUICKSORT(A, p, r)
1. IF p < r THEN
2.   q = PARTITION(A, p, r)
3.   QUICKSORT(A, p, q-1)
4.   QUICKSORT(A, q+1, r)
```

- Baseado em **divisão e conquista**, o algoritmo para ordenar $A[p, \dots, r]$ pode ser descrito por:
 - **DIVISÃO** – **Particionar o vetor $A[p, \dots, r]$ em dois subvetores potencialmente vazios $A[p, \dots, q-1]$ e $A[q+1, \dots, r]$ de forma que os elementos $A[p, \dots, q-1]$ sejam menores ou iguais a $A[q]$, que, por sua vez, é menor ou igual a todos os elementos em $A[q+1, \dots, r]$.**
 - **CONQUISTA** – **Ordenar $A[p, \dots, q-1]$ e $A[q+1, \dots, r]$ recursivamente, usando QUICKSORT.**

- **COMBINAÇÃO** – Como os subvetores $A[p, \dots, q - 1]$ e $A[q + 1, \dots, r]$ já estão ordenados, trabalho adicional não é necessário para combiná-los: o subvetor $A[p, \dots, r]$ está ordenado.

Particionamento

- O procedimento **PARTITION** é responsável pela fase de divisão do **QUICKSORT** e funciona reorganizando o subvetor $A[p \dots r]$ in loco. O algoritmo PARTITION apresentado a seguir é desenvolvido por Lomuto, e não foi o PARTITION original desenvolvido por Hoare.

```

PARTITION(A, p, r)
1.  x = A[r]
2.  i = p - 1
3.  FOR j = p TO r-1 DO
4.      IF A[j] <= x THEN
5.          i = i + 1
6.          troca A[i] e A[j]
7.      END IF
8.  END FOR
9.  troca A[i+1] e A[r]
10. RETURN i+1

```

- PARTITION sempre escolhe $x = A[r]$ como pivô, o elemento utilizado como divisor do subvetor. Enquanto executa, o procedimento particiona o vetor em **quatro regiões**. No início de cada iteração FOR das linhas 3–8, as regiões satisfazem certas propriedades. **Podemos utilizar estas propriedades como invariantes de laço para provar que PARTITION está correto.**

Correção do PARTITION

- Invariante de Laço – No início de cada iteração do laço “FOR” das linhas 3–8, para qualquer índice k ,
 - Se $p \leq k \leq i$, então $A[k] \leq x$
 - Se $i + 1 \leq k \leq j - 1$, então $A[k] > x$
 - Se $k = r$, então $A[k] = x$
- A Figura 1 abaixo mostra as quatro regiões que o vetor está particionado durante a execução de partition, bem como a relação entre as regiões, o enunciado da invariante de laço e as variáveis de controle do algoritmo de particionamento.

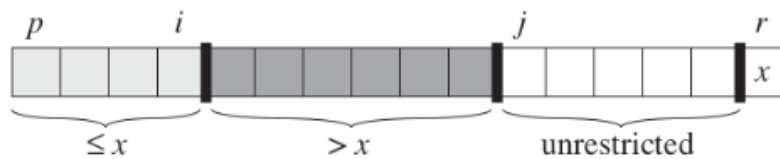


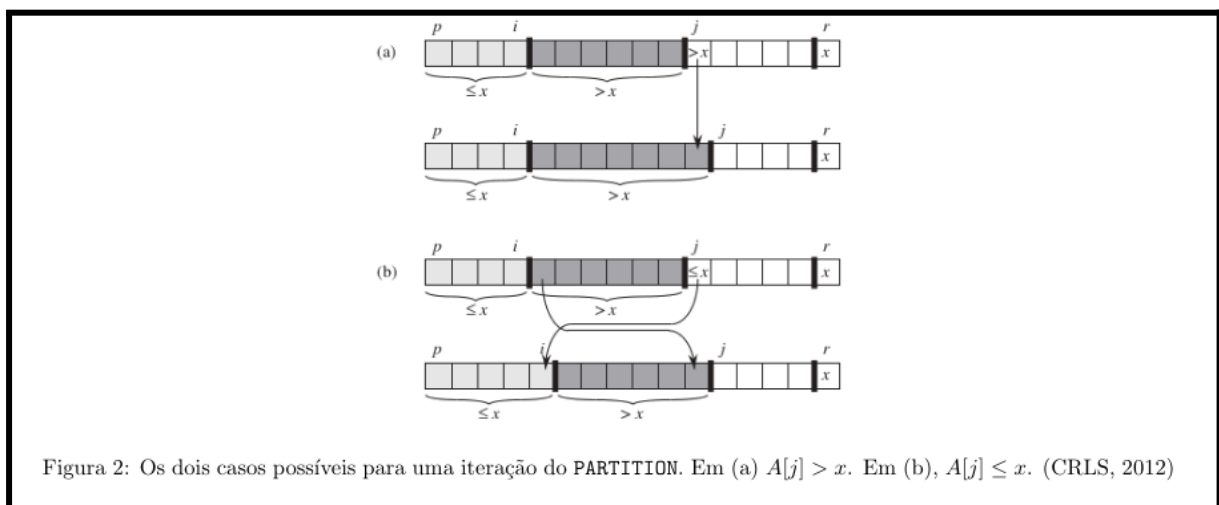
Figura 1: Representação gráfica da invariante de laço do algoritmo PARTITION (CRLS, 2012)

- Lembrando que os elementos em j e $r - 1$ não fazem parte dos casos e os valores destas relações não tem relação particular com o valor. Assim, precisamos mostrar que esta invariante de laço é verdadeira imediatamente antes da primeira iteração, que cada iteração do laço mantém a invariante válida para a próxima iteração e que a invariante provê uma propriedade útil para mostrar que o algoritmo está correto quando o laço termina.
- **INICIALIZAÇÃO** – Antes da primeira iteração do laço, $i = p - 1$ e $j = p$. Como não há elementos entre p e i e não há elementos entre $i + 1$ e $j - 1$, as duas primeiras propriedades são trivialmente verdadeiras. A atribuição da linha 1 satisfaz a terceira propriedade.
- **MANUTENÇÃO** – Consideremos os 2 casos possíveis: o caso onde a condicional da linha 4 é verdadeira e o caso falso. A Figura 2 ilustra o que acontece em ambos os casos. Quando $A[j] > x$, a única operação é incrementar j . Assim que j é incrementado a propriedade 2 é verdadeira para $A[j - 1]$ e as outras condições se mantêm inalteradas. Em outras palavras, o elemento sendo analisado pela condicional ($A[j]$) é um número maior que o pivô e já está na partição certa, uma vez que os elementos maiores que o pivô, por invariante de laço, estão entre $i + 1$ e $j - 1$.
- No caso $A[j] \leq x$, i é incrementado, troca $A[i]$ e $A[j]$ e então j é incrementado. Devido à troca, temos que $A[i] \leq x$, e a propriedade 1 é satisfeita. Da mesma forma, $A[j - 1] > x$, uma vez que o item que foi trocado para a posição $j - 1$ (que estava em $A[i]$, o primeiro maior que x após a instrução de incremento) é, por invariante de laço, maior que x , satisfazendo a segunda propriedade. A terceira propriedade continua verdadeira uma vez que x não é atribuído valores durante a execução do laço e que j nunca assume o valor r .

-

Desempenho do QuickSort

- O desempenho do QUICKSORT **depende do balanceamento obtido por PARTITION**, que depende da escolha do pivô e da relação dos valores dos elementos. Se o particionamento é balanceado, ou seja, o pivô “divide” o subvetor em 2 partições com quantidades praticamente iguais de elementos, o algoritmo possui complexidade assintótica idêntica ao HEAPSORT e MERGESORT. Caso contrário, a complexidade assintótica é a mesma que a do ineficiente BUBBLESORT. Primeiramente, analisamos informalmente o desempenho do QUICKSORT em relação ao balanceamento do particionamento.



Particionamento no Pior Caso

- O **pior caso do QUICKSORT** acontece quando PARTITION produz dois subproblemas: **um com 0 elementos e outro com $n - 1$ elementos em todas as chamadas recursivas em QUICKSORT**. Como o **particionamento tem custo $\Theta(n)$** e QUICKSORT(0) tem custo constante, a recorrência neste caso é:

$$\begin{aligned} T(n) &= T(n-1) + T(0) + \Theta(n) \\ &= T(n-1) + \Theta(n) \end{aligned}$$

- Intuitivamente, ao somarmos o custo de todos os níveis da recursão teríamos n níveis, cada um com um elemento a menos que o nível anterior de custo $\Theta(n)$ teríamos uma série aritmética ($cn + c(n-1) + c(n-2) + \dots + c1$), que é $\Theta(n^2)$. Desta forma, se o particionamento for totalmente desbalanceado em todo nível da recursão, **o tempo de execução é $\Theta(n^2)$** , que não é melhor que o INSERTIONSORT, por exemplo. Além disso, o pior caso ocorre quando o vetor de entrada já

está ordenado, um caso que o INSERTIONSORT executa em tempo $\Theta(n)$.

Particionamento no Melhor Caso

- No corte (particionamento) mais bem distribuído possível, **dois subproblemas devem ser resolvidos recursivamente, um com $\lceil n/2 \rceil$ elementos e outro com $\lceil n/2 \rceil - 1$** . Neste caso, o tempo necessário para QUICKSORT executar é dado pela recorrência

$$T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n)$$

- ignorando o chão e teto e a subtração de 1, sem perda de generalidade. Pelo caso 2 do teorema mestre, **a recorrência tem solução $T(n) = \Theta(n \lg n)$** . Portanto, **se o particionamento for balanceado em todos os níveis, temos um algoritmo mais eficiente, comparável ao HEAPSORT, com constantes escondidas potencialmente menores.**

Particionamento no Caso Médio

- **Uma das vantagens práticas do QUICKSORT é que a complexidade do caso médio se assemelha mais ao melhor caso do que ao pior caso.** Para entender melhor esta relação é necessário entender como o balanço do particionamento reflete na recorrência que descreve o tempo de execução.
- Suponha que **PARTITION sempre produza subproblemas em proporção 9 pra 1 em todos os níveis da recursão**. Note que esta aproximação é grosseira, uma vez que em uma execução média todo tipo de corte seria realizado por PARTITION, nas mais variadas proporções. No entanto, como veremos adiante, esta aproximação nos permite ter uma boa noção do tempo de execução no caso médio, sem nos ater aos cálculos mais complicados em uma contabilização mais detalhada da distribuição de probabilidade dos possíveis cortes realizados por PARTITION. **Portanto, supondo a proporção 9 pra 1, a recorrência de QUICKSORT seria**

$$T(n) = T\left(\frac{9n}{10}\right) + T\left(\frac{n}{10}\right) + \Theta(n)$$

- Como exercício, construa a árvore de recursão para esta recorrência. Note que todo nível da árvore tem custo cn até o nível de altura $\log_2 10 = \lg 10$. A partir deste nível, todos os demais possuem custo $O(cn)$, ou

seja, no máximo cn . A recursão termina na altura $\log_{10/9} n = \Theta(\lg n)$. É possível mostrar, pelo método da substituição que $T(n) = \Theta(n \lg n)$.

- Assim, com um balanceamento com corte em proporção 9 pra 1 em todo nível da recursão, que intuitivamente parece relativamente desbalanceado, QUICKSORT executa em $O(n \lg n)$, assintoticamente o mesmo que um corte perfeitamente balanceado em todas as chamadas recursivas. De fato, mesmo um corte em proporção 99 pra 1 em todas as chamadas levaria tempo $O(n \lg n)$. Aliás, qualquer corte de proporção constante leva a uma árvore de altura $\Theta(\lg n)$, tal que o custo de cada nível é $O(n)$. **Por fim, pode-se concluir que, intuitivamente, desde que o corte seja em proporção constante, o tempo de execução de QUICKSORT é $O(n \lg n)$.**